



МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Факультет Информационных технологий

Кафедра Кафедра информационных систем

Направление подготовки/Специальность Прикладная информатика

ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Старжинского Владислава Алексеевича

(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

Оглавление

ВВЕДЕНИЕ.....	2
ПОСТАНОВКА ЗАДАЧИ.....	2
1 РЕАЛИЗАЦИЯ ПРОГРАММНОГО РЕШЕНИЯ.....	3
1.1 Используемые средства разработки.....	3
1.2 Архитектура программы.....	3
1.3 Алгоритм обработки документов.....	4
1.4 Формирование итогового Excel-файла.....	5
2 ОЦЕНКА КАЧЕСТВА РАБОТЫ.....	6
ВЫВОД.....	7
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	7
ПРИЛОЖЕНИЕ А. ЛИСТИНГ ПРОГРАММЫ.....	8

ВВЕДЕНИЕ

Автоматическая обработка текстовых материалов является распространенной задачей при подготовке учебных, справочных и исследовательских наборов данных. На практике исходные сведения часто хранятся в разных форматах: часть материала находится в текстовых файлах, часть — в PDF-документах. Ручное извлечение определений из таких источников занимает много времени и может приводить к пропускам или ошибкам при переносе данных.

Цель работы — разработать программу на языке Python, которая извлекает определения из TXT- и PDF-файлов, выполняет предварительную очистку текста, классифицирует найденные формулировки по степени полноты и формальной записываемости, а затем формирует итоговый Excel-файл с извлеченными данными.

В работе используется алгоритмический подход без применения нейронных сетей. Основными средствами выступают стандартные модули Python для работы с файлами, регулярными выражениями и структурами данных, библиотека PyMuPDF для чтения PDF-документов и библиотека openpyxl для формирования файла Excel.

ПОСТАНОВКА ЗАДАЧИ

Необходимо реализовать программный модуль, который получает на вход набор текстовых и PDF-документов, находит в них определения и сохраняет результат обработки в табличном виде. Основной итоговый файл должен быть пригоден для дальнейшего просмотра, проверки, редактирования и использования в отчете.

Для выполнения работы требуется реализовать следующие функции:

- загрузка текстовых файлов из папки sources/texts;
- загрузка PDF-документов из папки sources/pdf;
- очистка исходного текста от лишних пробелов, переносов и служебных символов;
- поиск определений по явной разметке вида [D001], [D002] и далее;
- автоматический поиск определений по словам-маркерам при отсутствии явной разметки;
- выделение примерного термина из найденного определения;
- классификация определений по категориям SELF_CONTAINED, NEEDS_PRIOR_DEF и INFORMAL;
- формирование итогового Excel-файла Definitions_Review.xlsx;
- сохранение полного результата обработки в JSON-файл;
- формирование краткой сводки о запуске программы.

Результатом работы программы являются файл Definitions_Review.xlsx с извлеченными определениями, файл definitions_dataset.json с полной структурой записей и файл run_summary.json с краткой сводкой о количестве обработанных документов и распределении найденных классов.

1 РЕАЛИЗАЦИЯ ПРОГРАММНОГО РЕШЕНИЯ

1.1 Используемые средства разработки

Программа реализована на языке Python. Для обхода файловой структуры и работы с путями используется модуль `pathlib`. Для поиска определений и анализа текстовых признаков применяется модуль `re`, обеспечивающий работу с регулярными выражениями. Для хранения структурированных записей используются `dataclass`-объекты, позволяющие явно описать состав входных и выходных данных.

PDF-документы читаются с использованием библиотеки `PyMuPDF`. Она позволяет извлекать текст со страниц PDF-файла без ручного копирования содержимого. Итоговый файл Excel формируется с помощью библиотеки `openpyxl`, которая предоставляет средства создания рабочих книг, настройки ширины столбцов, форматирования заголовков и оформления таблицы.

В качестве итогового результата выбран формат `XLSX`, так как он удобен для просмотра, фильтрации, ручной проверки и последующей передачи результатов в отчетные материалы.

1.2 Архитектура программы

Программа построена в объектно-ориентированном стиле. Основная логика разделена на несколько классов, каждый из которых отвечает за отдельный этап обработки: загрузку данных, нормализацию текста, извлечение определений, классификацию и экспорт результатов.

Таблица 1 — Основные классы программы

Класс	Назначение	Основные элементы
SourceText	Хранение исходного документа после чтения.	<code>path</code> , <code>source_kind</code> , <code>content</code>
Candidate	Хранение найденного определения до итоговой сборки.	<code>item_id</code> , <code>definition_ru</code> , <code>source_file</code>
DatasetRecord	Итоговая запись для выгрузки в Excel.	<code>id</code> , <code>term</code> , <code>definition_ru</code> , <code>predicted_label</code>
TextNormalizer	Очистка текста от переносов, лишних пробелов и служебных символов.	<code>clean()</code>
SourceCollector	Сбор TXT- и PDF-документов из входных папок.	<code>collect()</code> , <code>_read_pdf()</code>
DefinitionExtractor	Поиск определений в исходном тексте.	<code>extract()</code>
TermExtractor	Выделение термина из формулировки определения.	<code>extract()</code>
RuleAnnotator	Автоматическая классификация определения.	<code>annotate()</code>
DatasetAssembler	Сбор итоговых записей на основе найденных кандидатов.	<code>assemble()</code>
Exporter	Сохранение результатов в XLSX, JSON и summary.	<code>write_xlsx()</code> , <code>write_json()</code> , <code>write_summary()</code>
Application	Общий сценарий запуска программы.	<code>build()</code>

Класс SourceCollector отвечает за получение исходных документов. Он проверяет наличие папок sources/texts и sources/pdf, после чего читает все подходящие файлы. Текстовые файлы считываются напрямую в кодировке UTF-8, а PDF-документы обрабатываются постранично.

Класс DefinitionExtractor реализует два способа поиска данных. Первый способ основан на явной разметке определений с идентификаторами вида [D001]. Второй способ используется как резервный: программа делит текст на предложения и выбирает те, где встречаются слова-маркеры, например «называется», «определение», «будем называть» или английские конструкции defined as и is called.

Класс RuleAnnotator выполняет автоматическую классификацию найденного определения. В нем заданы наборы признаков, указывающих на зависимость от внешнего контекста, неформальность формулировки или наличие формальных маркеров. На основании этих признаков каждой записи присваивается один из трех классов.

Общая схема обработки данных

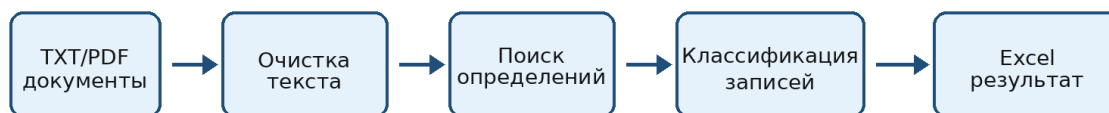


Рисунок 1 — Общая схема обработки данных

1.3 Алгоритм обработки документов

Алгоритм начинается с поиска входных файлов. Программа не требует обязательного наличия справочных файлов и работает только с папкой sources. Если исходные документы отсутствуют, формируется диагностическое сообщение о необходимости добавить TXT- или PDF-файлы.

После загрузки каждый документ проходит нормализацию. На этом этапе удаляются управляющие символы, мягкие переносы, лишние пробелы, повторяющиеся пустые строки и переносы слов через дефис. Такая предварительная обработка уменьшает количество ошибок при последующем поиске определений.

Далее программа извлекает определения. Если в документе есть блоки с идентификаторами [D001], [D002] и так далее, они используются как основные границы записей. Это наиболее надежный вариант, поскольку каждое определение уже явно отделено от соседнего текста. Если такая разметка отсутствует, программа применяет поиск предложений по словам-маркерам.

Для каждой найденной записи дополнительно выполняется попытка выделить термин. Для этого используются регулярные выражения, анализирующие начало формулировки. Например, из конструкции «Группой называется множество...» можно выделить термин «Группой». Если термин не удастся определить автоматически, соответствующее поле в итоговой таблице остается пустым.

После извлечения данных выполняется классификация. Если в тексте встречаются слова «ранее», «выше», «сохраняя обозначения» и другие признаки зависимости от контекста, запись получает класс NEEDS_PRIOR_DEF. Если обнаружены оценочные признаки вроде «немного», «много», «легко», «визуально», запись относится к классу INFORMAL. Если присутствуют формальные маркеры вроде «если», «для любого», «существует», знаки равенства или неравенства, запись относится к SELF_CONTAINED.

1.4 Формирование итогового Excel-файла

После сборки итоговых записей класс Exporter создает файл Definitions_Review.xlsx. В нем формируется лист Definitions, куда построчно записываются все найденные определения. Первая строка используется как заголовок таблицы и визуально выделяется цветом. Для удобства просмотра настраивается ширина столбцов и включается перенос текста внутри ячеек.

В итоговый Excel-файл включаются идентификатор записи, выделенный термин, текст определения, имя исходного файла, тип источника, предсказанная категория, признак возможности формализации, оценка уверенности правила, найденные признаки классификации, объяснение причины выбора категории и описание недостающего контекста.

Таблица 2 — Структура итогового Excel-файла

Столбец	Назначение
id	Идентификатор найденного определения.
term	Автоматически выделенный термин.
definition_ru	Полный текст найденного определения.
source_file	Имя исходного файла, из которого взята запись.
source_kind	Тип источника: txt или pdf.
predicted_label	Категория, присвоенная правилом классификации.
formalizable_now	Вывод о возможности формализации определения.
rule_score	Оценка уверенности правила.
evidence	Найденные текстовые признаки.
label_reason	Пояснение причины классификации.
missing_context	Описание данных, которых не хватает для строгой формализации.

Пример структуры итогового файла Definitions_Review.xlsx

id	term	definition_ru	source_file	predicted_label	formalizable_now
D001	Группа	Группой называется множество с операцией...	algebra.txt	SELF_CONTAINED	Да
D002	Кольцо	Кольцом называется множество с двумя операциями...	algebra.txt	SELF_CONTAINED	Да
P001		Ранее введенный объект будем называть...	lecture.pdf	NEEDS_PRIOR_DEFINITION	После восстановления контекста
P002		Неформальным определением считается...	notes.txt	INFORMAL	Нет

Рисунок 2 — Пример структуры итоговой таблицы

2 ОЦЕНКА КАЧЕСТВА РАБОТЫ

Качество работы программы оценивается не только по факту успешного запуска, но и по корректности каждого этапа обработки. Для проверки следует использовать набор документов, содержащих определения в явной разметке и без нее. Отдельно нужно проверить текстовые файлы и PDF-документы, так как механизм чтения этих форматов отличается.

Таблица 3 — Критерии оценки качества работы программы

№	Критерий	Проверка	Успешный результат
1	Чтение источников	Запуск с файлами в sources/texts и sources/pdf.	Файлы обнаружены и обработаны без ошибки.
2	Очистка текста	Просмотр definition_ru в Excel.	Нет лишних переносов, служебных символов и разрывов слов.
3	Извлечение определений	Сравнение Excel с исходным документом.	Основные определения попали в таблицу.
4	Классификация	Проверка поля predicted_label.	Категория соответствует смыслу определения.
5	Объяснение результата	Проверка evidence и label_reason.	Указана причина присвоения класса.
6	Excel-выгрузка	Открытие Definitions_Review.xlsx.	Таблица создана, столбцы читаемы, текст переносится.
7	Диагностика ошибок	Запуск без входных файлов или без определений.	Появляется понятное сообщение об ошибке.

Долю корректно извлеченных определений можно оценить по формуле: $P_{ext} = N_{ext} / N_{ref} \times 100 \%$, где N_{ext} — количество правильно найденных определений, а N_{ref} —

количество определений, которые должны быть извлечены из контрольного набора документов.

Качество классификации можно проверить вручную на небольшой выборке записей. Для этого эксперт просматривает значения `predicted_label` и сравнивает их с ожидаемым смыслом определения. Доля правильной классификации может быть рассчитана по формуле: $P_{label} = N_{correct} / N_{checked} \times 100 \%$, где `Ncorrect` — число верно классифицированных записей, а `Nchecked` — число проверенных записей.

Для проверки Excel-выгрузки необходимо открыть файл `Definitions_Review.xlsx` и убедиться, что все строки содержат исходный текст определения, имя файла-источника и результат классификации. Отдельно проверяется читаемость длинных формулировок: они должны переноситься внутри ячейки, а ширина столбцов должна позволять просматривать данные без постоянного ручного изменения структуры листа.

По результатам тестирования программа выполняет основные функции: читает исходные TXT- и PDF-файлы, извлекает определения, формирует объяснимую автоматическую классификацию и сохраняет результат в Excel-файл, пригодный для дальнейшей ручной проверки и включения в отчетные материалы.

ВЫВОД

В ходе работы разработана программа для извлечения определений из TXT- и PDF-документов с последующей выгрузкой результата в Excel. Реализованы загрузка исходных файлов, предварительная очистка текста, поиск определений по явной разметке и словам-маркерам, выделение термина, автоматическая классификация и сохранение результата в формате XLSX.

Программа построена в объектно-ориентированном стиле. Отдельные классы отвечают за сбор документов, нормализацию текста, извлечение определений, классификацию, сбор итоговых записей и экспорт результата. Такое разделение делает решение понятным и пригодным для дальнейшего расширения.

Качество результата оценивается по полноте извлечения определений, корректности классификации, наличию объясняющих признаков и правильности формирования Excel-файла. Разработанное решение может применяться для подготовки небольших учебных датасетов, анализа формулировок и первичной структуризации определений из текстовых материалов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Python. `pathlib` — Object-oriented filesystem paths [Электронный ресурс]. URL: <https://docs.python.org/3/library/pathlib.html> (дата обращения: 30.05.2026).
2. Python. `re` — Regular expression operations [Электронный ресурс]. URL: <https://docs.python.org/3/library/re.html> (дата обращения: 30.05.2026).
3. Python. `dataclasses` — Data Classes [Электронный ресурс]. URL: <https://docs.python.org/3/library/dataclasses.html> (дата обращения: 30.05.2026).
4. PyMuPDF Documentation [Электронный ресурс]. URL: <https://pymupdf.readthedocs.io/> (дата обращения: 30.05.2026).
5. openpyxl Documentation [Электронный ресурс]. URL: <https://openpyxl.readthedocs.io/> (дата обращения: 30.05.2026).
6. Microsoft. Excel file formats [Электронный ресурс]. URL: <https://learn.microsoft.com/en-us/deployoffice/compat/office-file-format-reference> (дата обращения: 30.05.2026).

ЛИСТИНГ ПРОГРАММЫ

Ниже представлен основной листинг программы main.py.

```
from __future__ import annotations

import argparse
import json
import re
from collections import Counter
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Iterable

LABELS = ("SELF_CONTAINED", "NEEDS_PRIOR_DEF", "INFORMAL")

@dataclass(frozen=True)
class SourceText:
    path: Path
    source_kind: str
    content: str

@dataclass(frozen=True)
class Candidate:
    item_id: str
    definition_ru: str
    source_file: str
    source_kind: str

@dataclass(frozen=True)
class DatasetRecord:
    id: str
    term: str
    definition_ru: str
    source_file: str
    source_kind: str
    predicted_label: str
    formalizable_now: str
    rule_score: float
    evidence: str
    label_reason: str
    missing_context: str

class TextNormalizer:
    @staticmethod
    def clean(text: str) -> str:
        text = re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f]", "", text)
        text = text.replace("\u00ad", "")
        text = re.sub(r"-\\s*\\n\\s*", "", text)
        text = re.sub(r"[ \\t]+", " ", text)
        text = re.sub(r"\n{3,}", "\n\n", text)
        return text.strip()

class SourceCollector:
    def __init__(self, source_dir: Path):
        self.text_dir = source_dir / "texts"
        self.pdf_dir = source_dir / "pdf"

    def collect(self) -> list[SourceText]:
        documents: list[SourceText] = []

        if self.text_dir.exists():
            for path in sorted(self.text_dir.glob("*.txt")):
                content = TextNormalizer.clean(path.read_text(encoding="utf-8"))
```

```

        documents.append(SourceText(path, "txt", content))

    if self.pdf_dir.exists():
        for path in sorted(self.pdf_dir.glob("*.pdf")):
            content = TextNormalizer.clean(self._read_pdf(path))
            documents.append(SourceText(path, "pdf", content))

    return documents

    @staticmethod
    def _read_pdf(path: Path) -> str:
        try:
            import pymupdf
        except ModuleNotFoundError as exc:
            raise RuntimeError("Для чтения PDF установите зависимость: pip install PyMuPDF") from exc

    parts: list[str] = []

    with pymupdf.open(path) as document:
        for page in document:
            parts.append(page.get_text("text", sort=True))

    return "\n".join(parts)

class DefinitionExtractor:
    block_pattern = re.compile(r"[(D\d{3})]\s*(.*) (?=(?:\n\s*\[(D\d{3})\])|\Z)", re.DOTALL)
    sentence_pattern = re.compile(r"(?<=[.!?])\s+")
    definition_marker = re.compile(
        r"(?i)(определени[ея]|называ(?:е|я|ются|ются)|будем называть|назов[её]м|defined as|is called)"
    )

    def extract(self, documents: Iterable[SourceText]) -> list[Candidate]:
        records: list[Candidate] = []
        generated_id = 1

        for document in documents:
            blocks = list(self.block_pattern.finditer(document.content))

            if blocks:
                for match in blocks:
                    text = TextNormalizer.clean(match.group(2))
                    records.append(Candidate(match.group(1), text, document.path.name,
document.source_kind))
                    continue

            for sentence in self.sentence_pattern.split(document.content):
                sentence = TextNormalizer.clean(sentence)

                if len(sentence) >= 35 and self.definition_marker.search(sentence):
                    records.append(Candidate(f"P{generated_id:03d}", sentence, document.path.name,
document.source_kind))
                    generated_id += 1

        return sorted(records, key=lambda value: value.item_id)

class TermExtractor:
    patterns = (
        re.compile(r"^(.{2,80}?)\s+называ(?:е|я|ются|ются)\s+", re.IGNORECASE),
        re.compile(r"^(.{2,80}?)\s+будем называть\s+", re.IGNORECASE),
        re.compile(r"^(.{2,80}?)\s+назов[её]м\s+", re.IGNORECASE),
        re.compile(r"^(определени[ея])\s+(.{2,80}?)\s+--", re.IGNORECASE),
    )

    @classmethod
    def extract(cls, text: str) -> str:
        normalized = re.sub(r"\s+", " ", text).strip()

```

```

for pattern in cls.patterns:
    match = pattern.search(normalized)

    if match:
        term = match.group(1).strip(" .,:;--")
        return term[:80]

return ""

class RuleAnnotator:
    dependent_terms = {
        "предыдущ": "ссылка на предыдущий фрагмент",
        "ранее": "ссылка на ранее заданный объект или условие",
        "введённ": "используется ранее введённый объект",
        "введенн": "используется ранее введённый объект",
        "выше": "ссылка на внешнее описание",
        "сохраняя обозначения": "не заданы используемые обозначения",
        "данной категории": "категория не описана в формулировке",
        "согласно выбранному": "используется ранее выбранная конструкция",
        "соглашениях раздела": "опора на соглашения вне определения",
        "зафиксированной ранее": "используется зафиксированный ранее объект",
        "обозначенного выше": "объект определён вне фрагмента",
    }

    vague_terms = {
        "неформальн": "явная отметка неформальности",
        "немного": "отсутствует числовая граница",
        "несуществен": "не задана допустимая величина изменения",
        "достаточно быстро": "не задан критерий скорости",
        "визуальн": "критерий зависит от восприятия",
        "много": "не задан порог количества",
        "легко": "не определена мера сложности",
        "незначитель": "не задана погрешность",
        "заметн": "оценочный признак",
        "почти всегда": "не задана вероятность или доля",
        "тесно": "не задано расстояние",
        "не слишком": "не задано ограничение",
        "очень редко": "не задана вероятность",
        "мало шагов": "не задан порог числа шагов",
        "особых свойств": "не перечислены свойства",
        "напоминает": "нет формального отношения",
        "пренебречь": "не задан допуск",
        "выглядит логично": "субъективная оценка",
        "серьёзно влияет": "не задана мера влияния",
    }

    formal_cues = (
        r"\если\b",
        r"\для любого\b",
        r"\для каждого\b",
        r"\существует\b",
        r"тогда и только тогда",
        r"∈",
        r"=",
        r"≤",
        r"≥",
        r"<",
        r">",
    )

    def annotate(self, text: str) -> tuple[str, str, str, str, float, str]:
        lower = text.lower()

        vague_hits = [{"term": {reason}} for term, reason in self.vague_terms.items() if term in lower]

        if vague_hits:
            evidence = "; ".join(vague_hits)
            return (
                "INFORMAL",
                "Нет",
                "Обнаружены качественные или субъективные критерии без точного условия.",
            )

```

```

        "Следует заменить оценочные слова числовым либо логическим условием.",
        0.96,
        evidence,
    )

    dependent_hits = [{"term": {reason}} for term, reason in self.dependent_terms.items() if term in
lower]

    if dependent_hits:
        evidence = "; ".join(dependent_hits)
        return (
            "NEEDS_PRIOR_DEF",
            "После восстановления контекста",
            "Формулировка зависит от обозначений или объектов, заданных вне найденного фрагмента.",
            "Необходимо добавить определения используемых объектов и условий.",
            0.93,
            evidence,
        )

    cues = [pattern for pattern in self.formal_cues if re.search(pattern, lower)]

    if cues:
        evidence = "формальные маркеры: " + ", ".join(cues)
        return (
            "SELF_CONTAINED",
            "да",
            "Объект и проверяемое условие заданы непосредственно в формулировке.",
            "Не требуется.",
            0.94,
            evidence,
        )

    return (
        "NEEDS_PRIOR_DEF",
        "После уточнения",
        "Не найдено явное проверяемое условие, достаточное для формальной записи.",
        "Требуется уточнить объект и критерий определения.",
        0.60,
        "формальные маркеры не обнаружены",
    )

class DatasetAssembler:
    def __init__(self):
        self.annotator = RuleAnnotator()

    def assemble(self, candidates: list[Candidate]) -> list[DatasetRecord]:
        records: list[DatasetRecord] = []

        for candidate in candidates:
            label, formalizable, reason, missing, score, evidence =
self.annotator.annotate(candidate.definition_ru)

            records.append(
                DatasetRecord(
                    id=candidate.item_id,
                    term=TermExtractor.extract(candidate.definition_ru),
                    definition_ru=candidate.definition_ru,
                    source_file=candidate.source_file,
                    source_kind=candidate.source_kind,
                    predicted_label=label,
                    formalizable_now=formalizable,
                    rule_score=score,
                    evidence=evidence,
                    label_reason=reason,
                    missing_context=missing,
                )
            )

        return records

```

```

class Exporter:
    def __init__(self, output_dir: Path):
        self.output_dir = output_dir
        self.output_dir.mkdir(parents=True, exist_ok=True)

    def write_json(self, records: list[DatasetRecord]) -> Path:
        path = self.output_dir / "definitions_dataset.json"
        path.write_text(
            json.dumps([asdict(record) for record in records], ensure_ascii=False, indent=2),
            encoding="utf-8",
        )
        return path

    def write_xlsx(self, records: list[DatasetRecord]) -> Path:
        if not records:
            raise RuntimeError("Нет данных для записи в Excel.")

        try:
            from openpyxl import Workbook
            from openpyxl.styles import Alignment, Font, PatternFill
            from openpyxl.utils import get_column_letter
            from openpyxl.worksheet.table import Table, TableStyleInfo
        except ModuleNotFoundError as exc:
            raise RuntimeError("Для формирования XLSX установите зависимость: pip install openpyxl") from exc

        workbook = Workbook()
        sheet = workbook.active
        sheet.title = "Definitions"

        headers = list(asdict(records[0]).keys())
        sheet.append(headers)

        for record in records:
            data = asdict(record)
            sheet.append([data[header] for header in headers])

        for cell in sheet[1]:
            cell.font = Font(bold=True, color="FFFFFF")
            cell.fill = PatternFill("solid", fgColor="1F4E78")
            cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)

        widths = {
            "A": 12,
            "B": 28,
            "C": 80,
            "D": 30,
            "E": 16,
            "F": 24,
            "G": 22,
            "H": 14,
            "I": 48,
            "J": 55,
            "K": 55,
        }

        for column, width in widths.items():
            sheet.column_dimensions[column].width = width

        for row in sheet.iter_rows(min_row=2):
            for cell in row:
                cell.alignment = Alignment(vertical="top", wrap_text=True)

        sheet.freeze_panes = "A2"

        end_row = len(records) + 1
        end_column = get_column_letter(len(headers))
        table = Table(displayName="DefinitionsTable", ref=f"A1:{end_column}{end_row}")
        table.tableStyleInfo = TableStyleInfo(
            name="TableStyleMedium2",

```

```

        showFirstColumn=False,
        showLastColumn=False,
        showRowStripes=True,
        showColumnStripes=False,
    )

    sheet.add_table(table)

    path = self.output_dir / "Definitions_Review.xlsx"
    workbook.save(path)

    return path

    def write_summary(self, records: list[DatasetRecord], documents: list[SourceText], excel_path: Path,
                      json_path: Path) -> Path:
        distribution = Counter(record.predicted_label for record in records)

        summary = {
            "theme": "Определения: полнота и формальная записываемость",
            "records_total": len(records),
            "source_files": {
                "txt": sum(document.source_kind == "txt" for document in documents),
                "pdf": sum(document.source_kind == "pdf" for document in documents),
            },
            "class_distribution": dict(distribution),
            "output": {
                "excel": str(excel_path),
                "json": str(json_path),
            },
        }

        path = self.output_dir / "run_summary.json"
        path.write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")

        return path

class Application:
    def __init__(self, root: Path):
        self.root = root

    def build(self) -> dict[str, object]:
        documents = SourceCollector(self.root / "sources").collect()

        if not documents:
            raise RuntimeError("Исходные файлы не найдены. Добавьте .txt в sources/texts или .pdf в sources/pdf.")

        candidates = DefinitionExtractor().extract(documents)

        if not candidates:
            raise RuntimeError("Определения не найдены. Используйте формат [D001] или текст со словами 'называется', 'определение', 'defined as'.")

        records = DatasetAssembler().assemble(candidates)

        exporter = Exporter(self.root / "output")
        json_path = exporter.write_json(records)
        excel_path = exporter.write_xlsx(records)
        summary_path = exporter.write_summary(records, documents, excel_path, json_path)

        return {
            "status": "ok",
            "records_total": len(records),
            "excel_file": str(excel_path),
            "json_file": str(json_path),
            "summary_file": str(summary_path),
        }

```

```

def main() -> None:
    parser = argparse.ArgumentParser(description="Извлечение определений из TXT/PDF и выгрузка результата в Excel.")
    parser.add_argument("--root", type=Path, default=Path(__file__).resolve().parent)

    args = parser.parse_args()
    result = Application(args.root).build()

    print(json.dumps(result, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```